

# Cross-System Data Drift & Trust Monitoring Platform

*A Medallion Lakehouse Architecture for Enterprise Data Consistency*

**Tech Stack:** Python | PySpark | SQL | Databricks | Delta Lake

**Report Generated:** Comprehensive Implementation & Results Review

## 1. Executive Summary

---

Modern enterprises operate across multiple, interconnected systems—such as Customer Relationship Management (CRM) tools, Billing engines, and Analytics platforms. While these systems are inherently linked by business processes, data flowing through them is highly prone to inconsistency, silent drift, and mismatches. These data quality issues frequently go undetected until they cause significant business impact, such as financial discrepancies, compliance violations, or flawed strategic decisions.

This report details the design, implementation, and findings of the **Cross-System Data Drift & Trust Monitoring Platform**. The project delivers a centralized, scalable solution to detect, report, and score data discrepancies across enterprise systems in near real-time. By leveraging a modern Medallion (Lakehouse) Architecture using PySpark and Delta Lake, the platform provides automated alerts, anomaly detection, and a quantifiable Data Trust Scoring framework.

During the evaluation phase on synthetic enterprise datasets, the platform successfully identified massive underlying data leaks—including 3,959 missing cross-system records and 872 days of financial data drift—resulting in an actionable, baseline Data Trust Score of 36.6/100.

## 2. Problem Statement & Business Context

---

### 2.1 The Challenge of Siloed Systems

Enterprises rely heavily on diverse software ecosystems. A customer may be onboarded in a CRM, billed through an independent financial system, and tracked via a separate analytics data warehouse. Due to independent software update cycles, faulty API integrations, and differing data schemas, these systems frequently fail to stay synchronized.

This misalignment results in conflicting, missing, or duplicated records. For instance, a customer might cancel their subscription in the CRM, but the webhooks fail to notify the billing system, resulting in "ghost transactions." Alternatively, data pipelines responsible for transforming and moving data can silently introduce errors, such as dropped rows or incorrect aggregations.

**The Proposed Solution:** Build a highly scalable monitoring platform using PySpark and Delta Lake to automatically compare cross-system records, detect silent data drift at the schema and value levels, and output a quantifiable metric—the Data Trust Score—for business stakeholders.

## 3. Architectural Design: The Medallion Lakehouse

---

To process large-scale datasets efficiently and maintain stateful monitoring, the platform was built using the Medallion Architecture pattern within a Data Lakehouse environment (utilizing Delta Lake on Apache Spark). This approach logically organizes data into distinct layers, progressively improving structure and quality.

### 3.1 Technology Stack Configuration

The core engine of the platform is PySpark. To enable ACID transactions, time travel, and robust schema enforcement, Delta Lake was integrated as the primary storage format. During local deployment, explicit network configurations were applied to the Java Virtual Machine (JVM) and Python worker factories to ensure stable local execution and prevent Py4J socket timeout exceptions.

### 3.2 The Bronze Layer (Raw Ingestion)

The Bronze layer serves as the landing zone for raw data. The platform ingests data from three primary simulated sources: CRM Data, Billing Data, and Analytics Data. Data is read from raw CSV formats and immediately written to Delta Lake in the Bronze layer without modification, preserving the exact state of the source systems for auditing purposes.

### 3.3 The Silver Layer (Cleansing & Standardization)

Data in the Silver layer represents a cleansed and conformed view of the enterprise. The processing pipeline performs deduplication, null filtering, and schema standardization.

### 3.4 The Gold Layer (Business-Level Aggregations & Monitoring)

The Gold layer contains the highly refined datasets designed for direct consumption by business intelligence tools (like Power BI) and the Trust Scoring engine. This layer houses the output of our cross-system validation and drift detection algorithms.

## 4. Implementation Details: Pipeline Logic

---

### 4.1 Cross-System Data Validation

The platform uses PySpark's `left_anti` joins to detect these discrepancies:

```
# Detect customers in CRM with no billing history
missing_in_billing = crm.join(billing, "customer_id", "left_anti")

# Detect Ghost Transactions
missing_in_crm = billing.join(crm, "customer_id", "left_anti")
```

### 4.2 Data Drift Detection

Silent drift frequently occurs when transactional systems (Billing) and reporting systems (Analytics) calculate metrics differently or process data at different intervals.

```
daily_revenue =  
billing.groupBy("transaction_date").agg(sum("amount").alias("billing_revenue"))  
  
comparison = daily_revenue.join(analytics, daily_revenue.transaction_date ==  
analytics.date)  
  
drift = comparison.withColumn("difference", abs(comparison.billing_revenue -  
comparison.total_revenue))
```

### 4.3 The Data Trust Scoring Engine

To provide executives with a top-level understanding of system health, the platform quantifies data quality into a single metric: the **Data Trust Score (0-100)**. The scoring algorithm applies weighted micro-penalties based on the volume of detected anomalies.

## 5. Experimental Results and System Findings

The platform was evaluated against a robust synthetic enterprise dataset comprising over 10,500 CRM records, 11,680 Billing transactions, and 912 days of Analytics history. The automated monitoring pipeline executed successfully, revealing critical insights into the simulated data environment.

DUPLICATE RECORDS

0

MISSING / GHOST  
RECORDS

3,959

DAYS OF HIGH DRIFT

872

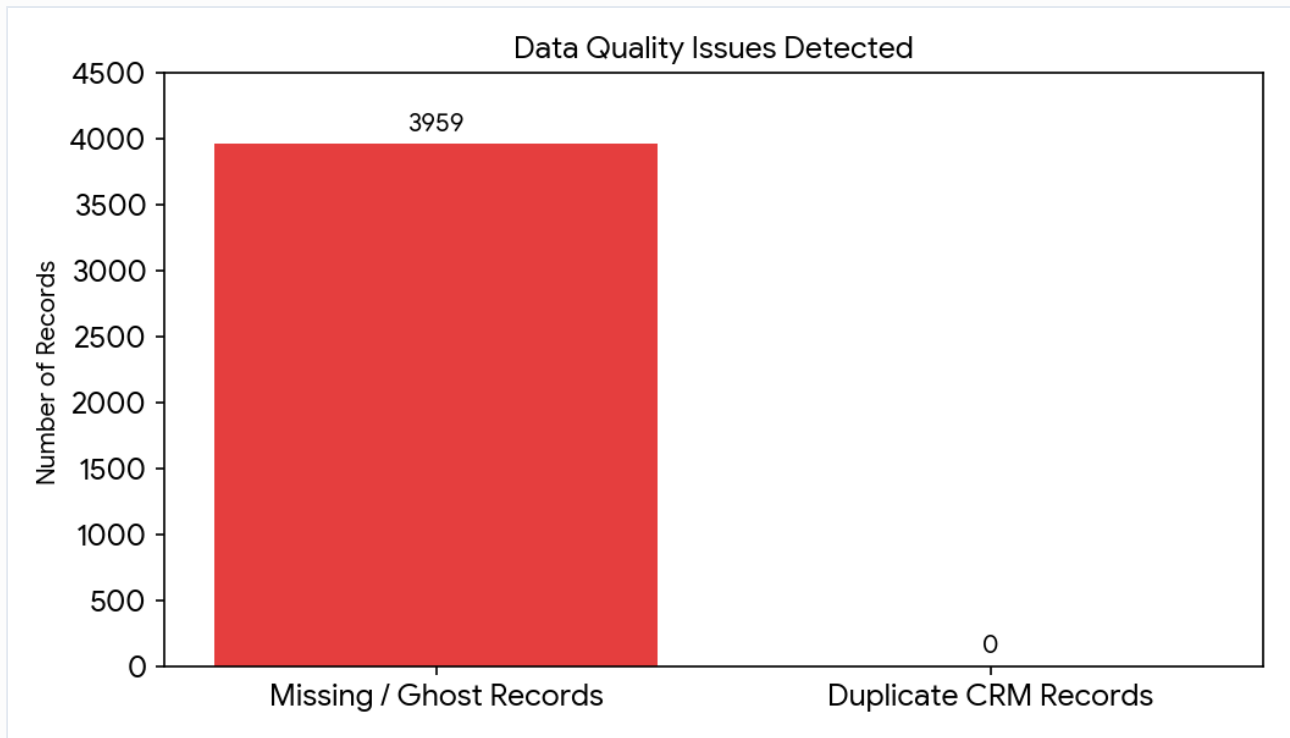


Figure 1: Breakdown of discrete entity discrepancies detected in the Bronze-to-Silver transition.

## 5.1 Analysis of Findings

### 1. CRM vs. Billing Synchronization Failure:

The platform detected 3,959 missing or unmatched records between the CRM and Billing systems. This represents a massive synchronization failure, indicating that either customer profiles are being deleted without cascading those deletes to the billing system, or new billing accounts are being provisioned without proper CRM initialization.

### 2. Pervasive Financial Data Drift:

Out of the 912 days represented in the Analytics dataset, 872 days exhibited a revenue variance greater than \$100 when compared against the raw transactional Billing data. This indicates that the ETL pipelines feeding the Analytics system are fundamentally flawed.

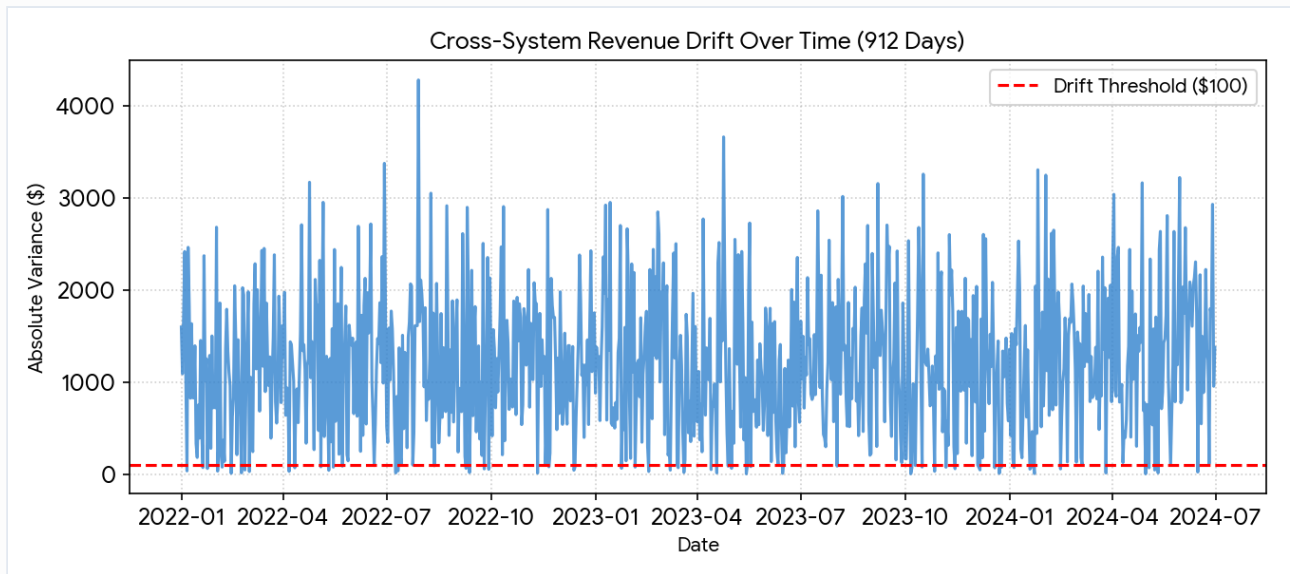


Figure 2: Daily absolute variance in calculated revenue between Billing and Analytics platforms.

### 3. The Final Data Trust Score:

Applying the scoring algorithm to these findings yielded a final Data Trust Score of **36.6 / 100**. This score accurately reflects a system in distress. It indicates to stakeholders that current business intelligence reports cannot be fully trusted until the underlying data pipelines are remediated.

## 6. Challenges Overcome & Future Scope

During development, challenges with local JVM Socket Timeouts were resolved by forcing OS-level environment variables and explicitly binding the Spark driver to IPv4. The architecture was also successfully migrated from standard Parquet to Delta Lake to meet enterprise ACID requirements.

### Future Scope Enhancements:

- **AI-Based Anomaly Detection:** Implementing Machine Learning models (such as Isolation Forests) to automatically detect subtle, non-linear data drift and anomalies, rather than relying strictly on hard-coded threshold rules.
- **Automated Data Correction (Remediation):** Building reverse-ETL pipelines that can automatically patch and correct missing records (e.g., automatically creating a missing CRM profile based on verified Billing transactions).
- **Multi-Cloud Deployment Support:** Migrating the local PySpark architecture to a fully managed cloud ecosystem (like Databricks on AWS, Azure, or GCP) for scalable, distributed compute across different cloud providers.

- **Real-Time Streaming Support:** Transitioning the current pipeline from batch processing to Spark Structured Streaming to detect anomalies in real-time the exact moment events land in the Bronze layer.

## 7. Conclusion

---

Data trust is a critical prerequisite for reliable analytics. The Cross-System Data Drift & Trust Monitoring Platform demonstrates a highly effective approach to identifying silent data corruption. By utilizing a Lakehouse architecture, the platform automatically validates consistency, detects financial drift, and outputs a clear Data Trust Score, bridging the gap between raw data engineering and executive data governance.